Explainable, Drift-Aware Serving of a Credit Card Fraud Detector: An End-to-End MLOps Case Study

Shamik Sengupta
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
email@vitstudent.ac.in

Yashaswini Kanaujia
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
email@vitstudent.ac.in

Sharmila C
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
email@vit.ac.in

***Abstract*—Fraud classifiers are usually judged by offline scores, yet in production an analyst also needs a reason for each alert, and the operator needs to know when live traffic stops resembling the training data. We describe an open-source pipeline that addresses all three needs for the public ULB credit card dataset (284,807 transactions, 0.17% fraud). Data is versioned with DVC, a gradient boosted tree model is trained with cost-sensitive weighting and tracked in MLflow, and the model is served through a typed FastAPI service. Every fraud alert carries its five largest TreeSHAP attributions, adding about 12 ms per flagged request. Input drift is computed by an Evidently worker that runs in a separate operating-system process, which removes a package conflict with the serving stack and keeps drift computation off the request path. Drift statistics are exported to Prometheus, alerted on and displayed in Grafana. On a stratified holdout, XGBoost reached a PR-AUC of 0.869 to 0.880 across configurations, against 0.719 for logistic regression and 0.858 for a random forest. A 20-trial Hyperopt search did not produce a statistically meaningful gain over default XGBoost settings. In a controlled test, the drift monitor flagged 2 of 29 features on unshifted traffic and 18 of 29 on shifted traffic, raising an alert only in the latter case. Median end-to-end latency was 27.5 ms.**

***Index Terms*—fraud detection, class imbalance, XGBoost, SHAP, data drift, MLOps, model monitoring**

I. Introduction

Card issuers score each authorisation request with a model before it is approved. Fraudulent transactions are a tiny minority, so a model that labels every request as genuine is right more than 99.8% of the time on our data while catching nothing. This is why the fraud literature has moved from accuracy towards precision, recall and the area under the precision-recall curve (PR-AUC) [3].

Two further requirements appear once a model leaves the notebook. First, the analyst who reviews an alert has to decide quickly whether to block a card. A bare probability gives no basis for that decision. Second, the population of transactions changes over time as merchants, customers and fraud tactics change [4]. Fraud labels typically arrive weeks after the transaction, when a chargeback is filed [2]. During that gap, a shift in the inputs is the earliest sign of trouble that is actually observable.

Each of these topics has a substantial literature, but they are usually studied in isolation. Systems papers note that most of the long-term cost of an ML system lies outside the model [5], and that monitoring is one of the steps practitioners most often report as difficult [6]. Integrating explanation and monitoring into a serving path also raises practical questions about latency, failure isolation and dependency management. These questions rarely appear in published work.

This paper reports how we built and evaluated such a system. Our contributions are:

- An open, reproducible pipeline that combines versioned data, leakage-free training, experiment tracking, a typed prediction API, per-alert SHAP explanations and automated input-drift alerting, with all components running under a single container orchestration file.
- A process-isolation design for drift computation that resolves an import-level conflict between the drift library and the web framework and keeps drift work off the request path.
- A baseline comparison with bootstrap confidence intervals, showing that cost-sensitive XGBoost outperforms linear and bagged baselines, while a 20-trial hyperparameter search adds no measurable benefit on this dataset.
- An empirical check of the drift monitor on unshifted and synthetically shifted traffic, together with latency measurements for the full service.

II. Related Work

A. Learning from Imbalanced Fraud Data

The ULB dataset was released together with a study of undersampling [1]. That study showed that undersampling biases the posterior probabilities and proposed a correction. Later work by the same group argued for evaluation protocols that reflect delayed labels and the limited number of alerts

an investigator can review [2]. Synthetic oversampling with SMOTE [7] is widely used, but it must be applied inside each training fold to avoid leakage. Gradient boosting with XGBoost [8] offers a simpler option: a class weight in the loss that requires no resampling. Hyperopt's Tree-structured Parzen Estimator [9] is a common choice for tuning such models.

B. Explaining Tree Ensembles

SHAP attributes a prediction to its input features using Shapley values, satisfying local accuracy and consistency [10]. For tree ensembles, TreeSHAP computes these values exactly in polynomial time [11]. This makes per-request explanation feasible inside an online service, unlike model-agnostic sampling methods.

C. MLOps and Monitoring

Kreuzberger et al. [12] identify versioning, CI/CD, experiment tracking, serving and monitoring as the core components of MLOps. MLflow [13] is a widely used tool for the tracking and packaging parts. Breck et al. [14] list tests that a production ML system should have, including tests of data, features and monitoring. For detecting distribution shift, Rabanser et al. [15] found that simple per-feature two-sample tests combined across features perform competitively against more complex detectors. Our monitor follows that approach.

III. System Design

A. Overview

Fig. 1 shows the four parts of the system: offline training, continuous integration, the serving application, and observability. All services are declared in one docker compose file. Kubernetes manifests are provided for cluster deployment.

B. Data and Training

The 150 MB source file is tracked with DVC, so a checkout resolves to the exact bytes identified by an MD5 hash. A single function performs a stratified 80/20 split with a fixed seed. This split is the only one in the codebase, and the test part is touched once, after model selection. Features V_1 to V_{28} are already PCA outputs and are passed through unchanged. *Time* and *Amount* are scaled with a median/IQR scaler, which is less sensitive to the heavy tail of transaction amounts. Both preprocessing steps live inside a scikit-learn `Pipeline`, so they are refitted on every cross-validation fold.

Class imbalance is handled in the loss rather than by resampling. With N_- legitimate and N_+ fraudulent training examples, XGBoost's positive-class weight is set to

$$w_+ = N_-/N_+ \approx 577. \tag{1}$$

Hyperopt searches nine hyperparameters (tree depth, learning rate, number of trees, row and column subsampling, γ, L_1 and L_2 penalties, and maximum delta step) over 20 trials. Each trial is scored by mean PR-AUC across five stratified folds of the training split. Every trial is logged as a nested MLflow run. The selected configuration is refitted on the full training split and stored with its holdout metrics.

C. Serving with Explanations

The API exposes single-transaction, form-style and batch CSV endpoints. The request body is a pydantic model with 30 required floating-point fields, generated from the same feature list the model uses, so a malformed request is rejected with HTTP 422 before inference. The service resolves the run with the highest holdout PR-AUC. It caches the loaded model and its explainer under that run identifier, rebuilding them only when a better run appears.

For a transaction classified as fraud, the service applies TreeSHAP to the booster after passing the row through the fitted preprocessing steps. The additive decomposition

$$g(\mathbf{x}) = \phi_0 + \sum_{j=1}^{30} \phi_j(\mathbf{x}) \tag{2}$$

holds in log-odds space, where g is the model margin and ϕ_0 is the expected margin. The response contains the five features with the largest $|\phi_j|$, each with its raw value, attribution and sign. Explanations are skipped for transactions predicted as legitimate, so their cost scales with the alert rate rather than with total traffic.

D. Isolated Drift Monitoring

Evidently 0.7 depends on a package that installs itself under the import name `multipart`. The FastAPI version we pin depends on a different package that claims the same name. The two cannot be imported in one interpreter. Upgrading the web stack to resolve this would have meant unpinning several serving dependencies. Instead, the serving process never imports Evidently. It only appends each request's features to a thread-safe ring buffer of size $N_w = 500$. When the buffer completes a window, a background task launches a short-lived worker process. The worker loads the conflicting module explicitly, compares the window with a 20,000-row reference sample drawn from the training split, writes a JSON summary and exits. The serving process reads the summary and updates the Prometheus gauges. A failure in the worker is logged and never propagates to the prediction path.

The monitor covers V_1 to V_{28} and *Amount*. *Time* is excluded because it counts seconds from the start of the dataset and would therefore always appear shifted on new data. For each feature j, the drift score is the normalised Wasserstein-1 distance between reference and current distributions,

$$W_1^{(j)} = \int_{-\infty}^{\infty} \left| F_{\text{ref}}^{(j)}(t) - F_{\text{cur}}^{(j)}(t) \right| dt, \tag{3}$$

and a feature is marked as drifted when $W_1^{(j)} > \delta = 0.1$. The drift share

$$s = \frac{1}{|\mathcal{F}|} \sum_{j \in \mathcal{F}} \mathbb{1}\left[W_1^{(j)} > \delta \right] \tag{4}$$

is exported together with the per-feature scores. Two Prometheus rules raise a warning when $s \geq 0.3$ holds for ten minutes.

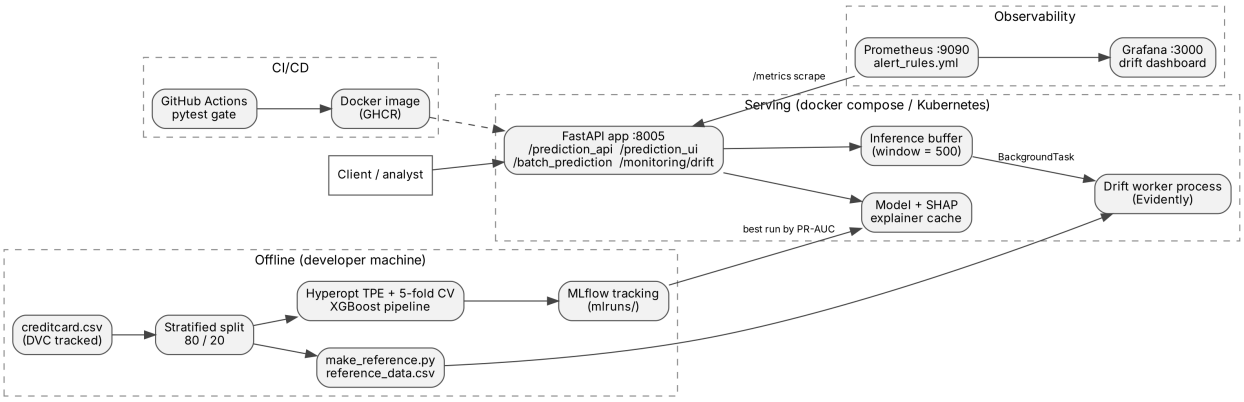


Fig. 1. System architecture. Training runs offline and writes to MLflow; CI builds the image only after tests pass; the API serves the best tracked run and buffers inputs; an isolated worker computes drift, which Prometheus scrapes and Grafana displays.

E. Testing, Packaging and CI

Fifteen automated tests cover schema validation, the feature contract, split invariants, SHAP output shape and ordering, the drift buffer, the parsing of drift summaries, gauge updates, and an end-to-end run of the drift worker. None of them needs the raw dataset or an MLflow server. The Docker build runs the full suite, and a GitHub Actions workflow runs it on every push, publishing the image to a container registry only if all tests pass.

IV. Experimental Setup

Data. The stratified split gives 227,845 training rows (394 frauds) and 56,962 test rows (98 frauds).

Models. We compare five models on the same split: logistic regression with standardised inputs and balanced class weights; a 200-tree random forest with balanced subsample weights; XGBoost with default parameters; XGBoost with default parameters and w_+; and the Hyperopt-tuned XGBoost described in Section III.

Metrics. PR-AUC (average precision) is the primary metric. We also report ROC-AUC, and precision, recall and F1 for the fraud class at a 0.5 threshold. Because the test set contains only 98 frauds, we estimate 95% confidence intervals for PR-AUC from 1,000 bootstrap resamples of the test set. We also use a paired bootstrap to compare the tuned and default weighted XGBoost models.

Drift protocol. We sent 300 unmodified test transactions through the API and triggered a drift check. We then sent 300 different test transactions in which *Amount* was transformed as $3a+50$ and twelve PCA components (V_1–V_7, V_9–V_{12}, V_{14}) were offset by $+1.5$, imitating a change in spending behaviour, and triggered a second check.

Environment. Python 3.10, scikit-learn 1.2.2, XGBoost 2.0.3, SHAP 0.44.1, MLflow 2.13.2, FastAPI 0.103, Evidently 0.7.21 and Prometheus 2.53. Training was performed on a laptop CPU, and the latency measurements were taken on a Linux machine.

TABLE I
Holdout Results (56,962 Transactions, 98 Frauds)

Model	PR-AUC [95% CI]	ROC	Prec.	Rec.	TP	FP
Logistic reg.	0.719 [0.63, 0.81]	0.972	0.061	0.918	90	1385
Random forest	0.858 [0.78, 0.93]	0.947	0.961	0.745	73	3
XGB default	0.878 [0.81, 0.94]	0.974	0.919	0.806	79	7
XGB + w_+	0.880 [0.81, 0.94]	0.968	0.882	0.837	82	11
XGB tuned	0.869 [0.80, 0.93]	0.981	0.863	0.837	82	13

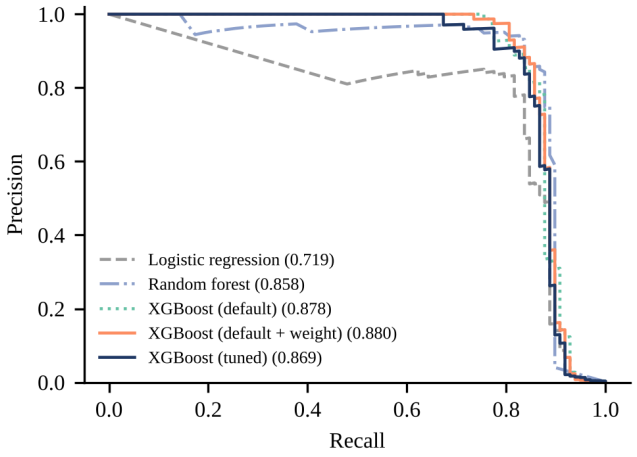


Fig. 2. Precision-recall curves on the holdout set. Average precision is given in brackets.

V. Results

A. Model Comparison

Table I and Fig. 2 show that the three XGBoost variants form the best group. Logistic regression attains a high ROC-AUC (0.972) and the highest recall. However, its precision at the default threshold is only 0.061, with 1,385 false alarms. This illustrates why ROC-AUC alone is misleading at this level of imbalance. The random forest is the most precise model (3 false alarms) but misses 25 of the 98 frauds.

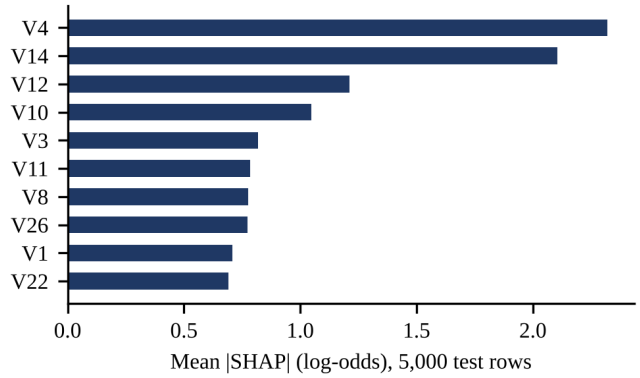


Fig. 3. Global importance of the tuned model, computed as the mean absolute TreeSHAP value over 5,000 holdout transactions.

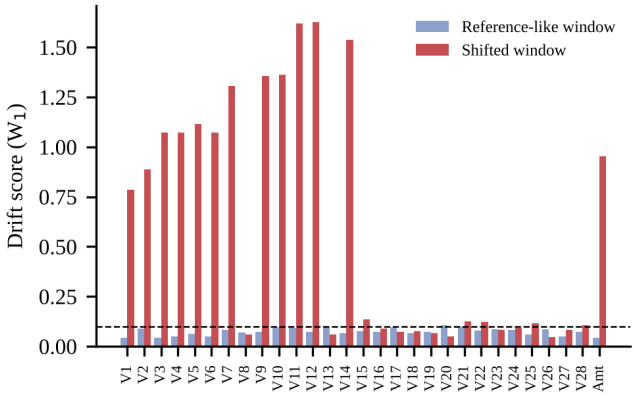


Fig. 4. Per-feature Wasserstein drift scores for an unshifted and a shifted window of 300 requests. The dashed line marks $\delta = 0.1$.

B. Effect of Hyperparameter Search

The 20 Hyperopt trials produced cross-validated PR-AUC values between 0.773 and 0.854. The best trial (depth 5, 300 trees, learning rate 0.093) reached 0.854. Default XGBoost reached 0.854 $\pm$ 0.014 under the same folds, and default XGBoost with w_+ reached 0.855 $\pm$ 0.021. On the holdout, the paired bootstrap difference between the tuned model and default weighted XGBoost was -0.011, with a 95% interval of $[-0.028, 0.004]$. The search therefore did not yield a measurable improvement. Wide ranges of hyperparameters appear to perform equally well on this dataset, and the main gains come from the choice of model family and from cost-sensitive weighting. The tuned model does have the highest ROC-AUC (0.981), and at a 0.5 threshold it matches the recall of the weighted default model (82 of 98 frauds) with two more false alarms.

C. Explanations

Fig. 3 aggregates per-transaction attributions into a global ranking, in which V_4, V_{14}, V_{12} and V_{10} dominate. For a fraudulent holdout transaction scored at $\hat{p} = 0.99997$, the API returned V_{14} ($\phi = +3.12$), V_{10} ($+1.75$) and V_{12} ($+1.23$) as the strongest contributions towards fraud, and V_8 (-0.92) and V_2 (-0.69) as contributions in the other direction. TreeSHAP added a median of 12.5 ms for each explained transaction.

D. Drift Detection

On unshifted traffic, two features (V_{20} and V_{21}) slightly exceeded δ, with scores of 0.104 and 0.101. This gives $s = 0.069$, well below the alert level. On shifted traffic, all 13 modified features scored between 0.78 and 1.63, and five unmodified features crossed δ by small margins, giving 18 drifted features and $s = 0.621$ (Fig. 4). Both alert rules entered the pending state on the next Prometheus scrape. The small false-positive count on unshifted windows is consistent with sampling noise at $N = 300$. The ten-minute hold on the alert rules is designed to absorb it.

E. Latency

Over 300 sequential requests, the median end-to-end latency of the explained prediction endpoint was 27.5 ms, with a 95th percentile of 39.4 ms. In-process inference took a median of 8.4 ms per row, and scoring the full 56,962-row holdout as one batch took 0.22 s. Drift computation takes several seconds, mostly for process start-up and library import, but it runs after the response has been returned and is therefore absent from these figures.

VI. Discussion and Limitations

The ULB dataset covers only two days. We therefore evaluated the drift monitor with a controlled shift rather than naturally occurring drift. Its sensitivity to subtler or gradual changes remains to be measured. Because the features are anonymised principal components, an explanation such as "V_{14} was unusually low" is faithful to the model but carries little meaning for a business user. On data with named features, the same mechanism would produce more actionable explanations. Input drift is also only a proxy: it neither guarantees nor rules out a drop in accuracy, and measuring accuracy over time requires the delayed labels this dataset cannot provide. Finally, the drift buffer is held in process memory, so it is not shared across replicas and is lost on restart.

VII. Conclusion

We have presented an end-to-end pipeline that turns a credit card fraud classifier into a service that explains its alerts and watches its own inputs. Cost-sensitive XGBoost clearly outperformed linear and bagged baselines in PR-AUC, while hyperparameter search gave no significant gain over default settings. Exact TreeSHAP explanations and process-isolated drift detection were added with modest latency and no coupling between the monitoring code and the serving path. Future work includes label-aware performance monitoring, a champion/challenger retraining loop triggered by sustained drift, and evaluation on longer, non-anonymised transaction streams.

REFERENCES

[1] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, 2015, pp. 159–166.

[2] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, 2018.

[3] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015.

[4] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, 2014.

[5] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 28, 2015, pp. 2503–2511.

[6] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Comput. Surv.*, vol. 55, no. 6, pp. 1–29, 2022.

[7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[9] J. Bergstra, D. Yamins, and D. Cox, "Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures," in *Proc. 30th Int. Conf. Mach. Learn. (ICML)*, 2013, pp. 115–123.

[10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

[11] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Mach. Intell.*, vol. 2, no. 1, pp. 56–67, 2020.

[12] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023.

[13] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.

[14] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *Proc. IEEE Int. Conf. Big Data*, 2017, pp. 1123–1132.

[15] S. Rabanser, S. Günnemann, and Z. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019.